1. **Code by Robbie that uses three scanner data to triangulate the 3D position of the chip(we might not be able to do this now, still need to try)**

```
import numpy as np

def find_position_with_tolerance(p1, p2, p3, r1, r2, r3, tolerance=0.1):
    # Ensure inputs are floats to avoid TypeError
    P1, P2, P3 = np.array(p1, dtype=float), np.array(p2, dtype=float), np.array(p3, dtype=float)
    r1, r2, r3 = float(r1), float(r2), float(r3)

    # 1. Local Coordinate System
    ex = (P2 - P1) / np.linalg.norm(P2 - P1)
    d = np.linalg.norm(P2 - P1)
    p3_p1 = P3 - P1
    i = np.dot(ex, p3_p1)
    ey = (p3_p1 - i * ex) / np.linalg.norm(p3_p1 - i * ex)
    j = np.dot(ey, p3_p1)
    ez = np.cross(ex, ey)

    # 2. Solve for local x and y
    x = (r1**2 - r2**2 + d**2) / (2 * d)
    y = ((r1**2 - r3**2 + i**2 + j**2) / (2 * j)) - ((i / j) * x)

    # 3. Handle the Z-squared tolerance
    z_sq = r1**2 - x**2 - y**2

    if z_sq < 0:
        # If the error is within our 0.1 tolerance, we assume the point
        # lies exactly on the plane (z=0)
        if abs(z_sq) < (tolerance**2):
            print(f"Note: Adjustment made within tolerance. z_sq was {z_sq:.4f}")
            z = 0
        else:
            print(f"Error: Spheres are too far apart (z_sq = {z_sq:.4f})")
            return None
    else:
        z = np.sqrt(z_sq)

    # 4. Global Transformation
    point_a = P1 + x * ex + y * ey + z * ez
    point_b = P1 + x * ex + y * ey - z * ez

    return point_a, point_b

# --- Testing

# change this stuff to actual input values.
pos1, r1 = (0, 13, 13), 18
pos2, r2 = (10, 0, 0), 10
pos3, r3 = (0, 0, 10), 10
```

```
result = find_position_with_tolerance(pos1, pos2, pos3, r1, r2, r3)
if result:
    print(f"Point A: {result[0]}\nPoint B: {result[1]}")
```

2. The server code that creates a server for all the data to collect the three scanner data

```
import math
from flask import Flask, request

app = Flask(__name__)

# -------------------------------
# Store latest RSSI values
# -------------------------------
latest_rssi = {
    "A": -70,
    "B": -70,
    "C": -70
}

# -------------------------------
# Scanner positions
# -------------------------------
A = (0, 0)
B = (3, 3)
C = (6, 0)

# -------------------------------
# Receive data from fakeData.py
# -------------------------------
@app.route("/data", methods=["POST"])
def receive_data():
    data = request.json

    scanner = data["scanner_id"]
    rssi = data["rssi"]

    latest_rssi[scanner] = rssi

    return "OK"

# -------------------------------
# RSSI → distance
# -------------------------------
def rssi_to_distance(rssi):
    tx_power = -59
    n = 2.5
    return 10 ** ((tx_power - rssi) / (10 * n))

# -------------------------------
# Trilateration
# -------------------------------
def trilaterate(d1, d2, d3):
```

```
    x1, y1 = A
    x2, y2 = B
    x3, y3 = C

    A1 = 2*(x2 - x1)
    B1 = 2*(y2 - y1)
    C1 = d1**2 - d2**2 - x1**2 + x2**2 - y1**2 + y2**2

    A2 = 2*(x3 - x1)
    B2 = 2*(y3 - y1)
    C2 = d1**2 - d3**2 - x1**2 + x3**2 - y1**2 + y3**2

    denom = A1*B2 - A2*B1

    if denom == 0:
        return (0, 0)

    x = (C1*B2 - C2*B1) / denom
    y = (A1*C2 - A2*C1) / denom

    return (round(x,2), round(y,2))

# -------------------------------
# Determine room
# -------------------------------
def get_room(x):
    if x < 3:
        return "C1-213"
    else:
        return "C1-212"

# -------------------------------
# Main page
# -------------------------------
@app.route("/")
def home():

    # convert RSSI → distance
    d1 = rssi_to_distance(latest_rssi["A"])
    d2 = rssi_to_distance(latest_rssi["B"])
    d3 = rssi_to_distance(latest_rssi["C"])

    # get position
    x, y = trilaterate(d1, d2, d3)

    # get room
    room = get_room(x)

    # map scaling (convert meters → pixels)
    scale = 80  # adjust if needed

    px = x * scale
    py = (3 - y) * scale  # flip y for screen

    # HTML
    html = f"""
```

```
<html>
<head>
   <meta http-equiv="refresh" content="5">
</head>

<body style="text-align:center;">

   <h1>BLE Tracking System</h1>

   <p>Position: ({x}, {y})</p>
   <p>Room: <b>{room}</b></p>

   <!-- Map container -->
   <div style="
      position: relative;
      width: 480px;
      height: 240px;
      margin:auto;
      border:2px solid black;
   ">

      <!-- Room divider -->
      <div style="
         position:absolute;
         left:240px;
         top:0;
         width:2px;
         height:240px;
         background:black;
      "></div>

      <!-- Room labels -->
      <div style="position:absolute; left:80px; top:10px;">C1-213</div>
      <div style="position:absolute; left:320px; top:10px;">C1-212</div>

      <!-- RED DOT -->
      <div style="
         position:absolute;
         left:{px}px;
         top:{py}px;
         width:12px;
         height:12px;
         background:red;
         border-radius:50%;
      "></div>

   </div>

   <br>

   <!-- Table -->
   <table border="1" style="margin:auto;">
      <tr>
         <th>Device</th>
         <th>Health</th>
         <th>Room</th>
```

```
            </tr>
            <tr>
                <td>Syringe Pump</td>
                <td>80%</td>
                <td>{room}</td>
            </tr>
        </table>

    </body>
    </html>
    """

    return html

# -------------------------------
# Run
# -------------------------------
if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

3. We are using one laptop to act as a scanner, the python code for this is:

```
import asyncio
import time
import requests
from bleak import BleakScanner

# -------------------------------
# CONFIG
# -------------------------------
TARGET_UUID = "E2C56DB5DFFB48D2B060D0F5A71096E0".replace("-", "").lower()
TARGET_MAJOR = 1
TARGET_MINOR = 2

SERVER_URL = "http://172.30.105.38:5000/data"
SCANNER_ID = "A"

# -------------------------------
# GLOBAL
# -------------------------------
latest_rssi = None


# -------------------------------
# BLE CALLBACK (FAST, NO BLOCKING)
# -------------------------------
def detection_callback(device, advertisement_data):
    global latest_rssi

    mfg = advertisement_data.manufacturer_data

    if 0x004C in mfg:
        data = mfg[0x004C]

        if len(data) >= 23 and data[0] == 0x02 and data[1] == 0x15:
            uuid_hex = data[2:18].hex()
```

```
        major = int.from_bytes(data[18:20], "big")
        minor = int.from_bytes(data[20:22], "big")

        if uuid_hex == TARGET_UUID and major == TARGET_MAJOR and minor == TARGET_MINOR:
            latest_rssi = advertisement_data.rssi

            # ✅ real-time print (safe)
            print(f"[LIVE] RSSI: {latest_rssi}")


# ------------------------------
# SEND LOOP (NON-BLOCKING FIX)
# ------------------------------
async def send_loop():
    global latest_rssi

    while True:
        await asyncio.sleep(5)

        if latest_rssi is None:
            print("[WAITING] No beacon yet...")
            continue

        payload = {
            "scanner_id": SCANNER_ID,
            "rssi": latest_rssi
        }

        try:
            # 🔥 RUN requests IN THREAD (IMPORTANT)
            response = await asyncio.to_thread(
                requests.post,
                SERVER_URL,
                json=payload
            )

            print(f"[SENT] {payload} | status={response.status_code}")

        except Exception as e:
            print("[ERROR sending]:", e)


# ------------------------------
# MAIN
# ------------------------------
async def main():
    scanner = BleakScanner(detection_callback)

    await scanner.start()
    print("Scanning started...")

    # run sending loop in parallel
    await send_loop()


asyncio.run(main())
```

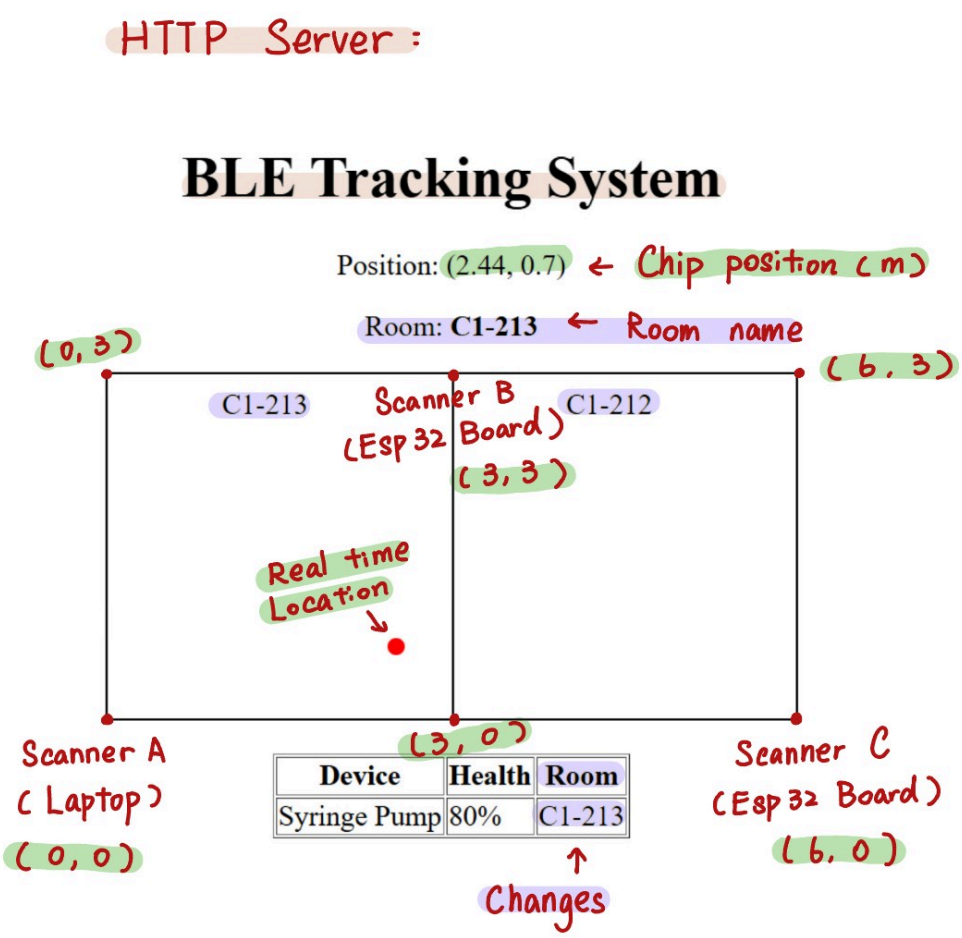
4. The arduino code for scanners

```
#include <WiFi.h>
#include <HTTPClient.h>
#include <BLEDevice.h>
#include <BLEScan.h>
#include <BLEAdvertisedDevice.h>
#include "esp_bt.h"

// ----------------------------
// WIFI CONFIG
// ----------------------------
const char* ssid = "Gillian";
const char* password = "13a2a3a3";

// ----------------------------
// SERVER CONFIG
// ----------------------------
const char* serverURL = "http://172.20.10.2:5000/data";
const char* scannerID = "B";

// ----------------------------
// TARGET BEACON
// ----------------------------
String targetUUID = "e2c56db5dffb48d2b060d0f5a71096e0";
uint16_t targetMajor = 1;
uint16_t targetMinor = 2;

// ----------------------------
// GLOBAL
// ----------------------------
volatile int latestRSSI = -100;
unsigned long lastSend = 0;
BLEScan* pBLEScan;

// ----------------------------
// BLE CALLBACK
// ----------------------------
class MyAdvertisedDeviceCallbacks: public BLEAdvertisedDeviceCallbacks {
    void onResult(BLEAdvertisedDevice advertisedDevice) {
```

```
        if (!advertisedDevice.haveManufacturerData()) return;

        // ✅ FIX: use Arduino String
        String str = advertisedDevice.getManufacturerData();
        int len = str.length();
        const uint8_t* data = (const uint8_t*)str.c_str();

        // Apple iBeacon
        if (len >= 25 && data[0] == 0x4C && data[1] == 0x00) {

            String uuid = "";
            char buf[3];

            for (int i = 4; i < 20; i++) {
                sprintf(buf, "%02x", data[i]);
                uuid += buf;
            }

            uint16_t major = (data[20] << 8) | data[21];
            uint16_t minor = (data[22] << 8) | data[23];

            if (uuid == targetUUID && major == targetMajor && minor ==
targetMinor) {
                latestRSSI = advertisedDevice.getRSSI();

                Serial.print("[LIVE RSSI] ");
                Serial.println(latestRSSI);
            }
        }
    }
};

// ----------------------------
// SETUP
// ----------------------------
void setup() {
    Serial.begin(115200);

    // free memory
```

```
    esp_bt_controller_mem_release(ESP_BT_MODE_CLASSIC_BT);

    // ---------------- WIFI ----------------
    WiFi.mode(WIFI_STA);
    WiFi.setSleep(false);
    WiFi.begin(ssid, password);

    Serial.print("Connecting to WiFi");

    while (WiFi.status() != WL_CONNECTED) {
        delay(500);
        Serial.print(".");
    }

    Serial.println("\n✅ Connected!");
    Serial.print("IP: ");
    Serial.println(WiFi.localIP());

    delay(1000);

    // ---------------- BLE ----------------
    BLEDevice::init("");

    pBLEScan = BLEDevice::getScan();
    pBLEScan->setAdvertisedDeviceCallbacks(new
MyAdvertisedDeviceCallbacks());

    // stable settings
    pBLEScan->setActiveScan(false);
    pBLEScan->setInterval(100);
    pBLEScan->setWindow(50);

    // start first scan
    pBLEScan->start(2, false);
}

// ----------------------------
// LOOP
// ----------------------------
void loop() {
```

```
// restart scan if stopped
if (!pBLEScan->isScanning()) {
    pBLEScan->start(2, false);
}

// send every 5 seconds
if (millis() - lastSend > 5000) {
    lastSend = millis();

    if (latestRSSI == -100) {
        Serial.println("[WAITING] No beacon yet");
        return;
    }

    int rssiToSend = latestRSSI;
    latestRSSI = -100;

    // check WiFi
    if (WiFi.status() != WL_CONNECTED) {
        Serial.println("[WARNING] WiFi lost, reconnecting...");
        WiFi.begin(ssid, password);
        return;
    }

    HTTPClient http;

    if (http.begin(serverURL)) {

        http.addHeader("Content-Type", "application/json");

        String json = "{\"scanner_id\":\"";
        json += scannerID;
        json += "\",\"rssi\":";
        json += rssiToSend;
        json += "}";

        int httpResponseCode = http.POST(json);

        Serial.print("[SENT] ");
```

```
            Serial.print(json);
            Serial.print(" | code: ");
            Serial.println(httpResponseCode);

            http.end();
        } else {
            Serial.println("[ERROR] HTTP begin failed");
        }

        delay(100);
    }
}
```

5.The server page